JC4004 – Computational Intelligence

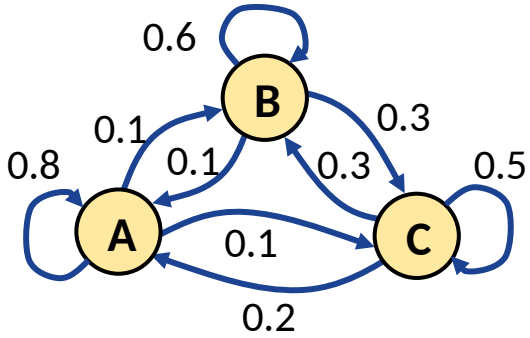
Sessions 7 & 8: Hidden Markov models

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)
Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)
Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

Nov-Dec 2024

Markov models

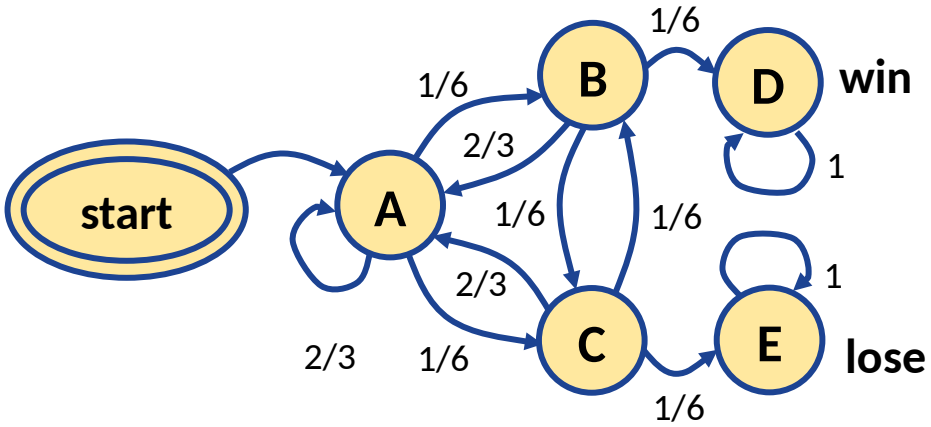
- A common method for modeling a stochastic processes with a limited number of possible states is to use a *Markov model*
  - The system has a group of possible states , and transitions between the states happen at certain probabilities at each time step
  - Transition probabilities can be written as a *transition matrix* , where rows represent source states and columns represent target states



$$P = \begin{bmatrix} 0.8 & 0.1 & 0.1 \\ 0.1 & 0.6 & 0.3 \\ 0.2 & 0.3 & 0.5 \end{bmatrix}$$

Markov model example

- Let us play a game, where you roll a dice: if you get two times six in a row, you win; if you get one two times in a row, you lose; otherwise, the game continues



$$P = \begin{bmatrix} \frac{2}{3} & \frac{1}{6} & \frac{1}{6} & 0 & 0 \\ \frac{2}{3} & 0 & \frac{1}{6} & \frac{1}{6} & 0 \\ \frac{2}{3} & \frac{1}{6} & 0 & 0 & \frac{1}{6} \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Distribution of states in Markov models

- Assuming Markov model with state space , we can define the *distribution* of the Markov chain at time as:

$$\mu_t(x) = Pr(X_t = x), x \in S$$

- Probability that the Markov model is in state at time , conditional to the state at time , can be defined as:

$$Pr(X_{t+1} = y) = \sum_{x \in S} Pr(X_t = x) Pr(X_{t+1} = y \vee X_t = x)$$

Distributions from transition matrix

- From the definitions, we can also express distribution as function of transition probabilities:

$$\mu_{t+1}(y) = \sum_{x \in S} \mu_t(x) P(x, y), x, y \in S$$

- Assuming and are row vectors, indexed according to the state space , we can also use matrix form:

$$\boldsymbol{\mu_{t+1} = \mu_t P}$$

Distribution at specific time

- Given arbitrary time , where , the distribution is:

$$\mu_t = \mu_0 P^t$$

- *Proof:* by definition, is the identity matrix, so , and:

$$\mu_{t+1} = \mu_t P = (\mu_0 P^t) P = \mu_0 (P^t P) = \mu_0 P^{t+1}$$

- To define how many times the system is expected to be in state during time interval , we can define *frequency of state* as :

$$N_t(y) = \sum_{s=0}^{t} Pr\ (X_s = y)$$

Occupancy matrix

- If the initial state of the model is , we can define the elements of the *occupancy matrix* as:

$$M_t(x,y)=E\left[N_t(y)\middle|X_0=x\right], x, y \in S$$

- Rows in represent the frequency distributions for each initial state
- Occupancy matrix can also be computed from transition matrix:

$$\boldsymbol{M}_t = \sum_{i=0}^{t} \boldsymbol{P}^i$$

Limiting distribution

- Concerning long term evolution of distributions, it is interesting to know if there exist *limiting distribution*
- We can assume that is the *equilibrium distribution* of Markov chain represented by transition matrix , if it fulfills:

$$\sum_{x \in S} \pi(x) P(x, y) = ¿ \pi(y) \forall y \in S ¿$$

- Or the same in matrix form: $\pi P = \pi$
- *If there is a limiting distribution, it is also the equilibrium distribution!*

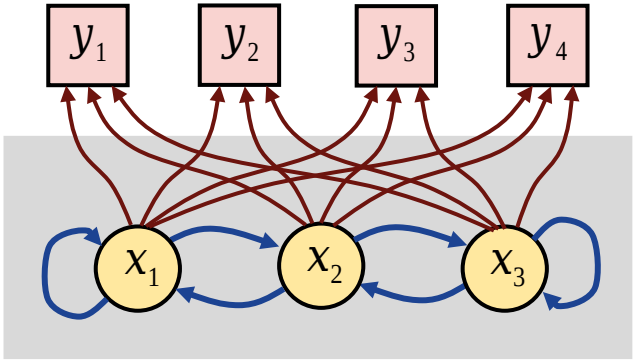
Solving equilibrium distribution

- It can be difficult to solve from , but is easy to solve from the system of equations
- Since is a probability distribution, is subject to condition
- ***Example:*** given $P = \begin{bmatrix} 0.6 & 0.4 & 0 \\ 0.3 & 0.5 & 0.2 \\ 0 & 0.6 & 0.4 \end{bmatrix}$, we get system of equations:

From the equations, unique solution can be found: , ,

Hidden Markov models

- Many practical applications use *hidden Markov models (HMM)*
  - The states of the underlying Markov model cannot be observed directly, but only through another “depending” process
  - The *observable states* depend on the states of the HMM at certain probabilities



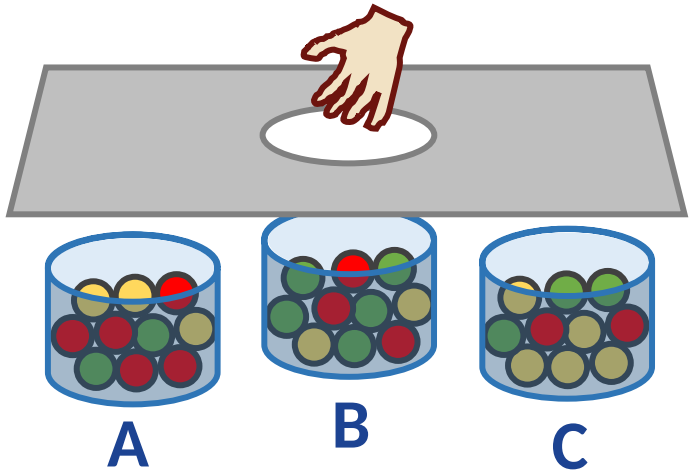
is known

$$Pr(y) = \sum_{x \in A} Pr(y \vee x) Pr(x¿), Pr(y \vee x) ¿$$

is unknown

Ball and urn HMM example

- Urns with different number of balls with different colours change places randomly (urns represent the hidden states)
  - Observed state is the color of the randomly picked ball



Observed state *Y* probabilities

	Y=red	*Y*=green	*Y*=yellow
X=A	0.5	0.2	0.3
X=B	0.3	0.5	0.2
X=C	0.2	0.3	0.5

Definitions for HMM

- State space of states for the Markov model :

  ; for example,
- Transition probabilities, i.e., Markov model transition matrix :
- Set of observed symbols (states) :

  ; for example,
- Observed symbol probability matrix :
- Initial state distribution:

Evaluation of HMM output probability

- *Problem:* how to compute the probability of observed sequence , given that parameters are known?
- *Solution:* decompose the problem by summing the probabilities for all the possible hidden state sequences

$$Pr(O|\lambda) = \sum_{a \ll Q} Pr(O|Q,\lambda)\, Pr(Q \vee ¿\, \lambda)\, ¿$$

Probability of generating sequence *O* from sequence of states *Q*

Probability of the model going through the sequence of states *Q*

Evaluation of HMM output probability

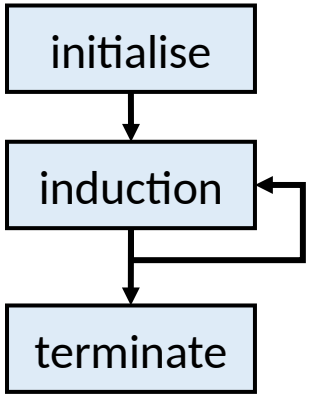
- Possible observation sequence for a given state sequence :

$$Pr(O|Q,\lambda)=\prod_{t=0}^{T} Pr\left(o_t \vee q_t,\lambda\right)=b_{q_0,o_0} b_{q_1,o_1} \cdots b_{q_T,o_T}$$

- Using the chain rule:

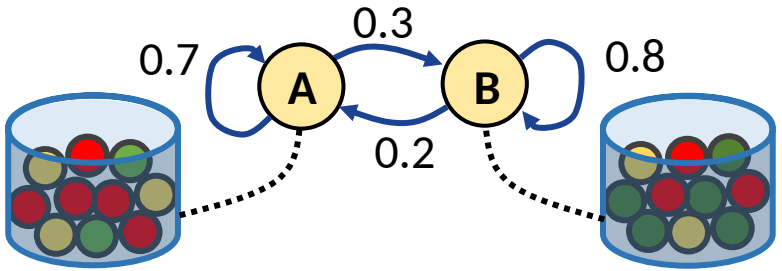
Forward algorithm walk-through

- We can solve the probability for by using *forward algorithm*
- The probability of seeing the sequence of observations and ending in state is: $\alpha_t(i) = Pr\left(O = \{o_0, o_1, \ldots, o_t\}, q_t = s_i \vee \lambda\right)$
- Initialisation: $\alpha_0(i) = \pi_i$
- Induction:
- Termination:



Example of forward algorithm

- Assume ball and urn scenario as shown here
- Assume observed sequence and



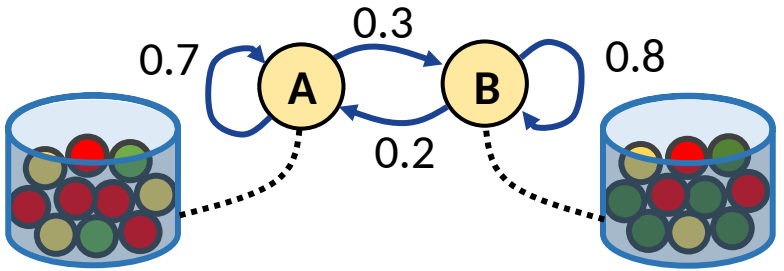
R R Y

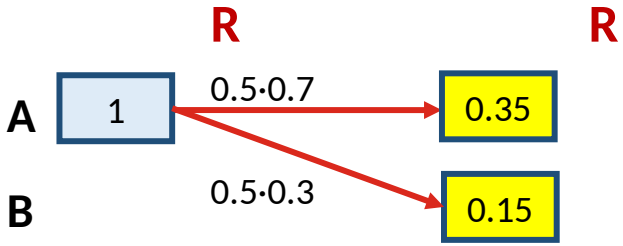
A 1

B

Example of forward algorithm

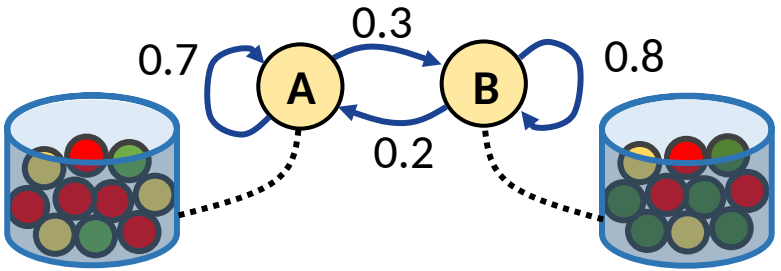
- Assume ball and urn scenario as shown here
- Assume observed sequence and

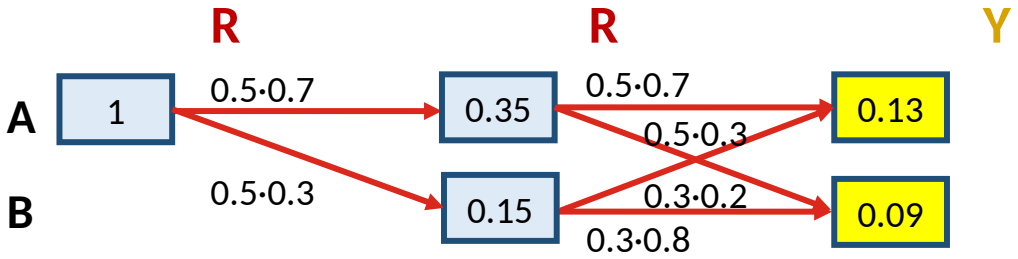




Example of forward algorithm

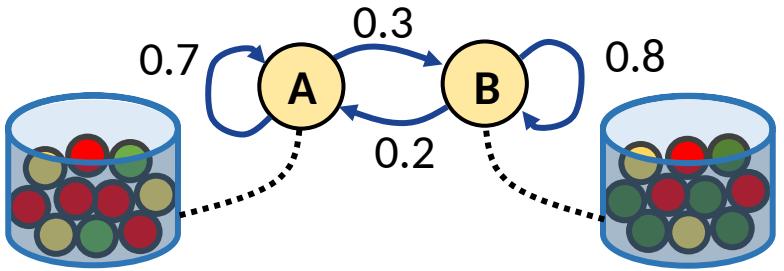
- Assume ball and urn scenario as shown here
- Assume observed sequence and

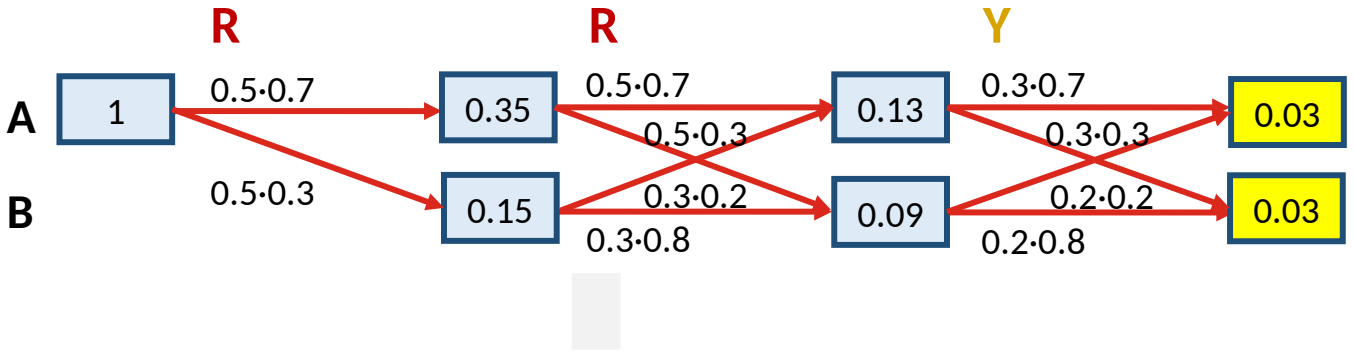




Example of forward algorithm

- Assume ball and urn scenario as shown here
- Assume observed sequence and





Break: any questions?

Viterbi algorithm

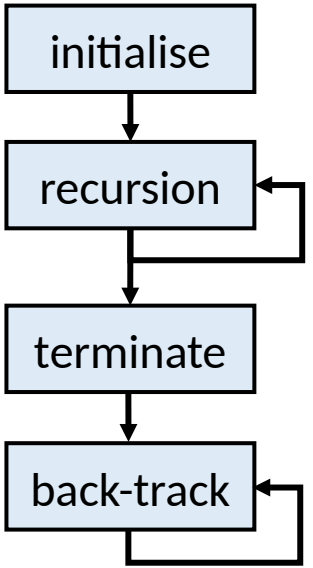
- In practical algorithms, we are often more interested in solving the most probable sequence of hidden states
- *Viterbi algorithm* can be used to solve , given the sequence of observed states and
- Let us assume that the last hidden state at time is , and define:

$$\delta_t(i) = \max_{q_0 q_1 \cdots q_{t-1}} Pr\left(¿\, q_0 q_1 \cdots q_{t-1}, q_t = s_i, o_0 o_1 \cdots o_t \vee \lambda\right) ¿$$

- And in the following transition: $\delta_{t+1}(j) = \max_i [\delta ¿¿ t(i) a_{i,j}] b_{j,o_{t+1}} ¿$

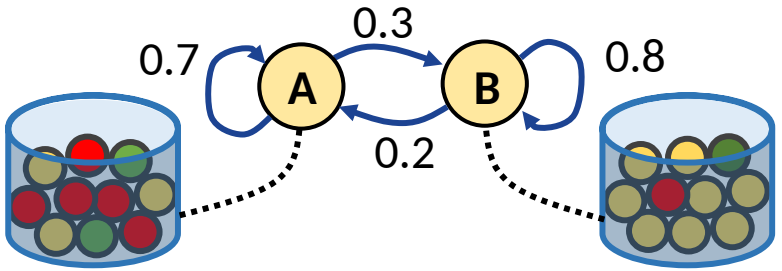
Viterbi algorithm walk-through

- Initialisation: $\delta_0(i) = \pi_i b_{i,o_0}; \psi_0(i) = 0$
- Recursion:
- Termination:
- Back-tracking: $q_t^* = \psi_{t+1}(q_{t+1}^*)$



Example of Viterbi algorithm

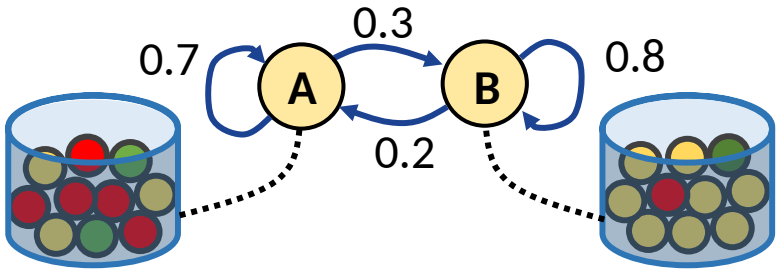
$$A = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{bmatrix} \qquad \Pi = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

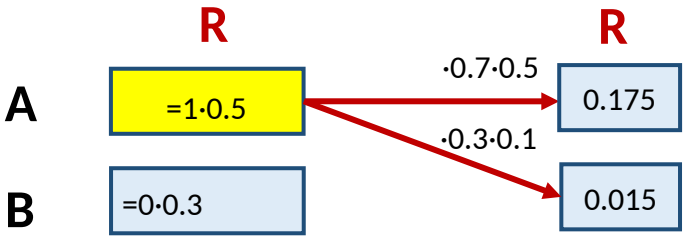


	R	R	Y
A	=1·0.5		
B	=0·0.3		

Example of Viterbi algorithm

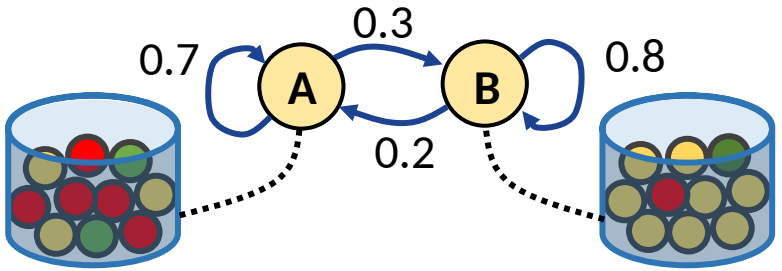
$$A = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{bmatrix} \qquad \Pi = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

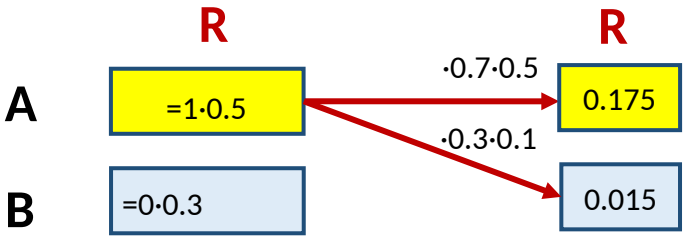




Example of Viterbi algorithm

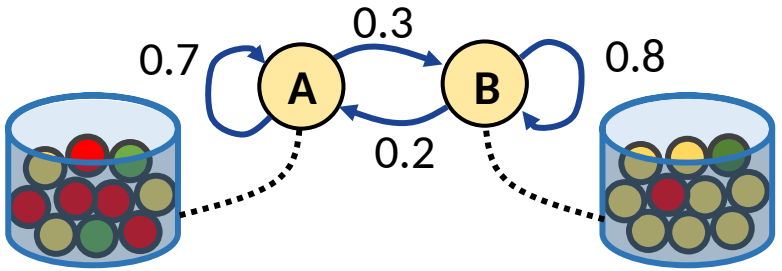
$$A = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{bmatrix} \qquad \Pi = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

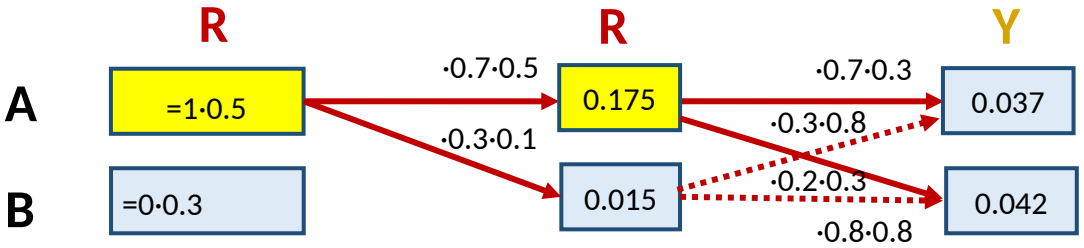




Example of Viterbi algorithm

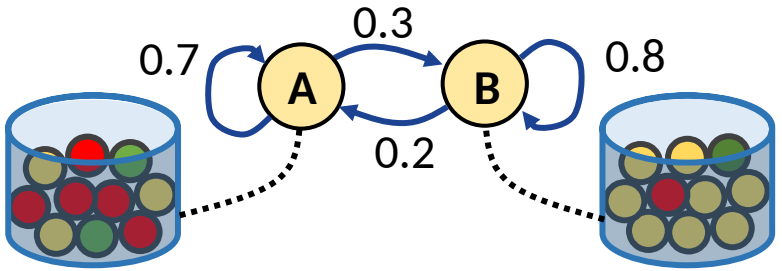
$$A = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{bmatrix} \qquad \Pi = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

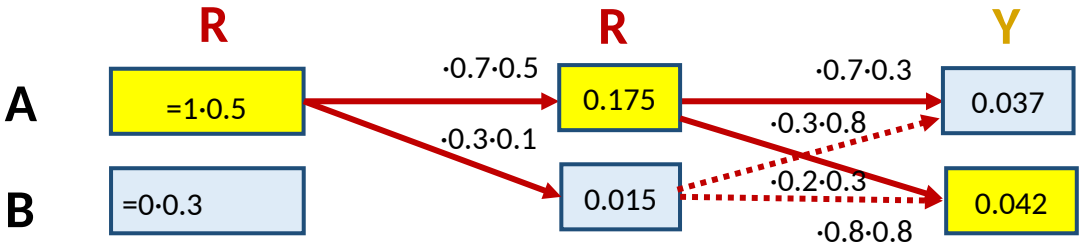




Example of Viterbi algorithm

$$A = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{bmatrix} \qquad \pi = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

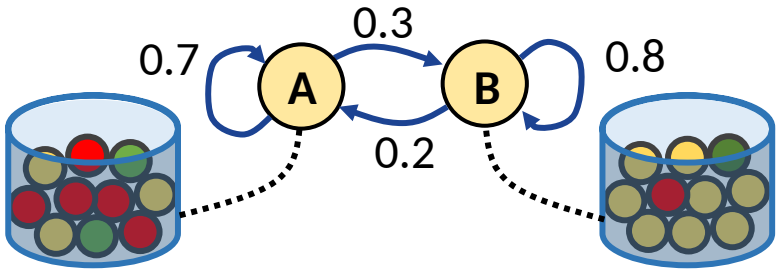




$$P^* = 0.042$$
$$q_2^* = B$$

Example of Viterbi algorithm

$$A = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{bmatrix} \qquad \Pi = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

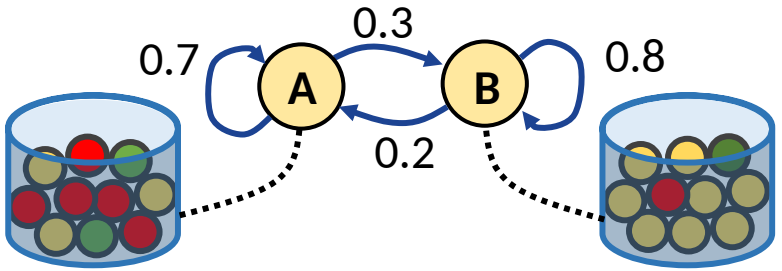


	R	R	Y
A	=1·0.5	0.175	0.037
B	=0·0.3	0.015	0.042

$$q_2^* = B$$

Example of Viterbi algorithm

$$A = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{bmatrix} \qquad \Pi = \begin{bmatrix} 1 & 0 \end{bmatrix}$$



	R	R	Y
A	=1·0.5	0.175	0.037
B	=0·0.3	0.015	0.042

$$q_2^* = B$$

$$Q = \{A, A, B\}$$

Learning the HMM parameters

- For using the Viterbi algorithm, we need to know the HMM model parameters, i.e., $\lambda = \{A, B, N, M, \pi\}$
  - However, what if λ is not known, and we want to find it out?
- Let us assume that the state space sizes n and m are known, so the problem is to find the most likely $\lambda = \{A, B, \pi\}$
  - Solution: find
  - Sequence of observations is considered as *training data*
  - Function/value is considered as *likelihood*
  - We want to find out the *maximum likelihood estimate* (MLE) of the model parameters , given some training data

Maximising likelihoods

- From the Bayes rule, we know that
  - Observed sequence is fixed, so it is enough to maximize
- We cannot usually estimate
  - We may assume that all the models are equally likely, or that models in some subclass are equally likely, and other models are irrelevant
  - Therefore, we could simply ignore and just optimize
- The most probable model is found by asking which model makes the sequence of observations most likely

Iterative learning algorithm

- For complex HMMs, optimal cannot be found analytically, but we can compute for a known , using the forward algorithm
- We can define a general iterative procedure to find a better model, given old model , as:
  1) Start with an initial model (e.g., randomly chosen initial state and random non-zero values for elements in and)
  2) Compute , based on and observations
  3) If is larger than the predefined threshold, update and return to step 2
  4) Stop and return the solution

Baum-Welch algorithm

- Baum-Welch algorithm is a learning algorithm designed especially for learning the parameters of HMMs
  1) Initialize , and (random/best guess)
  2) *Forward process:* compute parameters for all time steps and hidden state indices
  3) *Backward process:* compute parameters for all time steps and hidden state indices
  4) Compute temporary variables and
  5) Update HMM parameters , and , using values of γ and ξ
  6) Repeat from step 2 until desired level of convergence is reached

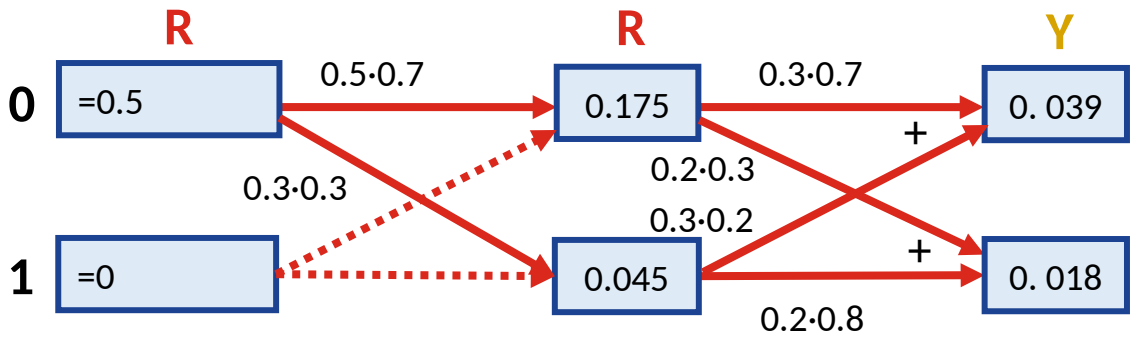
Forward process

- Definition of :
- Initialization:
- Induction:

Example:

$$A = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{bmatrix}$$

$$\Pi = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

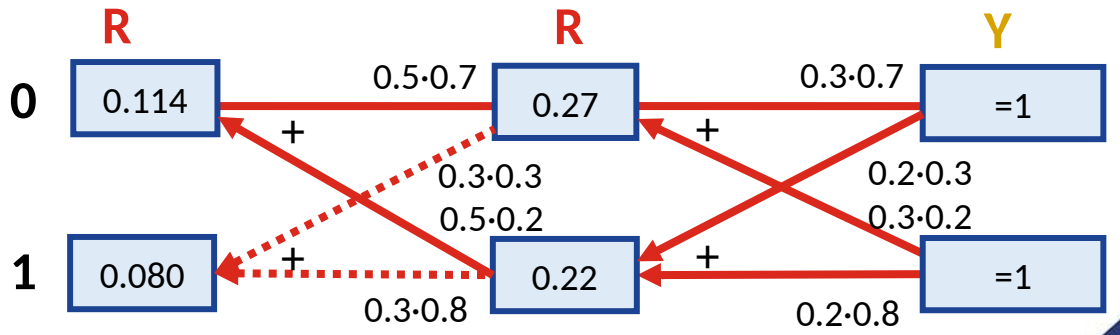


Backward process

- Definition of :
- Initialization:
- Induction:

Example:

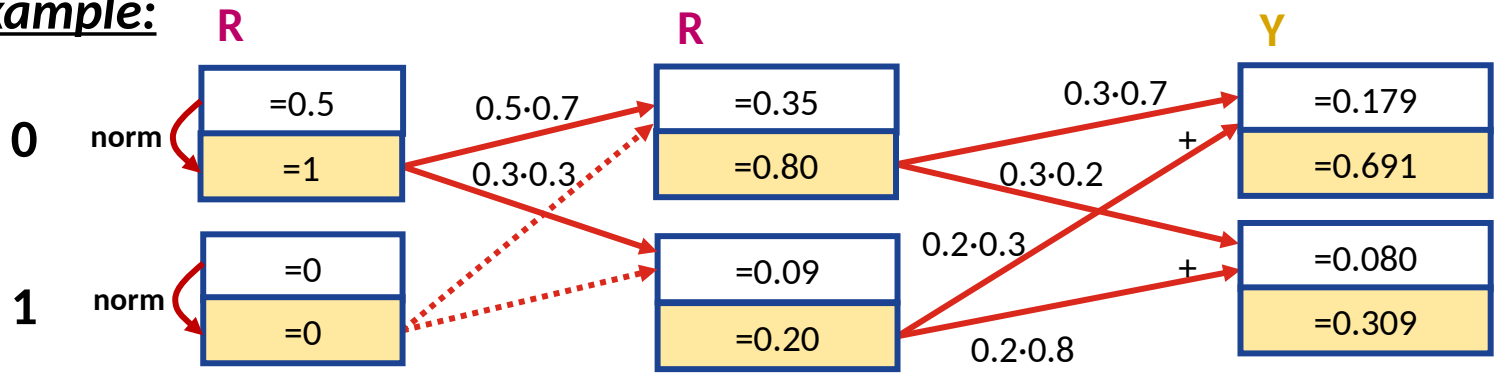
$$A = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{bmatrix}$$



Normalisation

- The problem of computing α and β is that their values get smaller and smaller when *t* gets larger, which may lead to underflow
- Values can be normalized as:

Example:



Update temporary variables

- Temporary variables and are updated as:

Update HMM parameters

- HMM parameters , and are updated as:

$$=$$

$$=, \quad \text{where}$$

- If there are multiple training sequences, we can compute temporary variables and for each sequence and use their averages

Summary

- *Markov models* are state-based models with transitions between states taking place at discrete time steps
  - Markov model state probabilities depend only on the previous state and the fixed transition probabilities
- *Hidden Markov model (HMM)* consists of a Markov model with hidden states and dependent observable states
  - HMMs can be trained to model different kinds of sequential processes, and HMMs have been used in many different applications
  - *Baum-Welch algorithm* is a special case of iterative training process, designed for training HMMs

Questions and discussion